

面向 LLM / Agent Systems 科研的深度学习零基础课

第 08 讲

为什么仅有 Linear 不够

本讲目标：用手算与实验真正证明，多层 Linear/Affine 之间若没有非线性操作，函数表达能力仍可折叠为一层；理解为什么 Activation Function 是深度神经网络的关键组成之一。

讲义完整版

本课程采用 Just-in-Time Math：数学只讲到足以理解深度学习，并通过手算 + NumPy + 后续 PyTorch 建立底层直觉。

目录

1	这一讲解决什么问题？	3
1.1	本讲最低验收标准	3
2	【原理】先从最简单的一维例子开始	4
3	【通俗解释】像把两次“直尺变换”合成一次	4
4	【必要数学】Function Composition 函数组合	5
5	【具体数字例子】二维 -> 三维 -> 二维	5
6	【公式逐项拆解】Shape 为什么也完全对得上？	6
7	Bias 会不会让它变成“非线性”？	7
8	【图解】为什么“层多”还不够	7
9	为什么这限制了表达能力？	7
10	那非线性到底改变了什么？	8
11	【代码实验】用 NumPy 把结论跑出来	9
12	【实验结果】加入 ReLU 后为什么折叠失败	10
13	【常见误区】	10
13.1	误区 1：神经网络只要层数多就一定更强	10
13.2	误区 2：Bias 已经让网络非线性	11
13.3	误区 3：ReLU 是分段直线，所以它仍然算 linear	11
13.4	误区 4：既然两层可折叠，那么训练两层和训练一层完全一样	11
14	Bottleneck：多层 Linear 仍可能有结构约束	11
15	【与 LLM / Transformer 的关系】	11
15.1	Transformer FFN 为什么是 Linear -> Activation -> Linear	11
15.2	现代 LLM 常见 SiLU / SwiGLU	12
15.3	不要误解为“Transformer 的非线性只来自 FFN”	12
15.4	Agent Systems 为什么需要懂？	12

16	【科研视角】Ablation 到底在测什么?	12
17	本讲完整心智链	13
18	本讲检查点	13
19	下一讲预告	14

1 这一讲解决什么问题？

上一讲我们已经把一个神经元写成：

$$z = xW + b$$

如果一层不够，我们自然会想到：那就再堆一层。

例如：

$$h = xW_1 + b_1$$

$$y = hW_2 + b_2$$

看起来网络已经“变深”了。但一个关键问题是：**如果中间什么都不做，只是继续 Linear，真的会得到新的函数能力吗？**

本讲核心结论

不会。若相邻层之间只有 Linear/Affine 变换，没有非线性操作，那么这些层可以代数上合并成一个新的 Linear/Affine 变换。

$$(xW_1 + b_1)W_2 + b_2 = x(W_1W_2) + (b_1W_2 + b_2)$$

因此“层数更多”本身并不自动带来非线性表达能力。

与 LLM / Transformer / Agent Systems 的连接

你以前在 LLM 课程中见过 Transformer FFN/MLP: Linear -> GELU/SiLU -> Linear。今天要回答的正是：为什么中间那个 Activation 不能随便删掉。没有它，FFN 的两个 Linear 在函数上可以折叠成一个 Linear。

1.1 本讲最低验收标准

学完后你应该能：

1. 手算两个一维 affine 变换如何合并；
2. 推出两层矩阵形式的折叠公式；
3. 解释 Bias 为什么不会改变“可折叠”结论；
4. 解释“深度”与“非线性表达能力”不是同一件事；
5. 看懂为什么 ReLU/GELU/SiLU 会阻止简单线性折叠；
6. 区分“函数表达能力相同”和“训练过程完全相同”；
7. 连接 Transformer FFN 中的 Linear + Activation + Linear。

2 【原理】先从最简单的一维例子开始

第一层：

$$h = 2x + 1$$

第二层：

$$y = 3h - 4$$

把第一层直接代入第二层：

$$y = 3(2x + 1) - 4$$

展开：

$$y = 6x + 3 - 4 = 6x - 1$$

于是原来的两层：

$$x \rightarrow 2x + 1 \rightarrow 3h - 4$$

和一层：

$$y = 6x - 1$$

对所有 x 完全一样。

用数字检查

取 $x = 2$ 。两层：

$$h = 2 \times 2 + 1 = 5$$

$$y = 3 \times 5 - 4 = 11$$

折叠后：

$$y = 6 \times 2 - 1 = 11$$

结果一致。

3 【通俗解释】像把两次“直尺变换”合成一次

一个 affine 变换可以先理解成：

- 拉伸或压缩；
- 改变方向；
- 混合不同坐标；
- 再整体平移。

如果第一步仍然只是这种变换，第二步仍然只是这种变换，那么两步合起来，最终仍然属于同一类变换。

直觉

“两次拉伸/旋转/平移”可以重新计算出“一次等效的拉伸/旋转/平移”。

真正改变函数类别的，是中间加入一个不能被吸收到矩阵乘法里的**非线性操作**。

4 【必要数学】Function Composition 函数组合

这是本讲唯一需要补的一点数学。

若：

$$f(x) = 2x + 1$$

$$g(h) = 3h - 4$$

那么：

$$g(f(x)) = 3(2x + 1) - 4$$

这个过程叫 **Function Composition (函数复合)**。

在神经网络里，网络其实就是很多函数一层层复合：

$$y = f_L(f_{L-1}(\cdots f_2(f_1(x))))$$

问题就在于：如果所有 f_i 都只是 affine，那么复合以后仍然是 affine。

5 【具体数字例子】二维 -> 三维 -> 二维

现在不再只看一个数字。

设：

$$X : (B, 2)$$

第一层：

$$W_1 : (2, 3), \quad b_1 : (3,)$$

因此：

$$H = XW_1 + b_1 : (B, 3)$$

第二层：

$$W_2 : (3, 2), \quad b_2 : (2,)$$

因此：

$$Y = HW_2 + b_2 : (B, 2)$$

代入 H ：

$$Y = (XW_1 + b_1)W_2 + b_2$$

利用矩阵乘法分配律：

$$Y = X(W_1W_2) + b_1W_2 + b_2$$

定义：

$$W_{eq} = W_1W_2$$

$$b_{eq} = b_1W_2 + b_2$$

于是：

$$Y = XW_{eq} + b_{eq}$$

两层被精确变成了一层。

6 【公式逐项拆解】Shape 为什么也完全对得上？

原始两层：

$$W_1 : (2, 3), \quad W_2 : (3, 2)$$

所以：

$$W_1W_2 : (2, 2)$$

这正好就是折叠后：

$$W_{eq} : (2, 2)$$

Bias：

$$b_1 : (3,)$$

乘：

$$W_2 : (3, 2)$$

得到：

$$b_1W_2 : (2,)$$

再加：

$$b_2 : (2,)$$

最终：

$$b_{eq} : (2,)$$

两层 affine 折叠公式

对本课程一直使用的 row-vector/Batch 记法：

$$(XW_1 + b_1)W_2 + b_2 = X(W_1W_2) + (b_1W_2 + b_2)$$

因此：

$$W_{eq} = W_1 W_2$$

$$b_{eq} = b_1 W_2 + b_2$$

7 Bias 会不会让它变成“非线性”？

不会。

严格数学里：

$$y = Wx$$

叫 linear map；

$$y = Wx + b$$

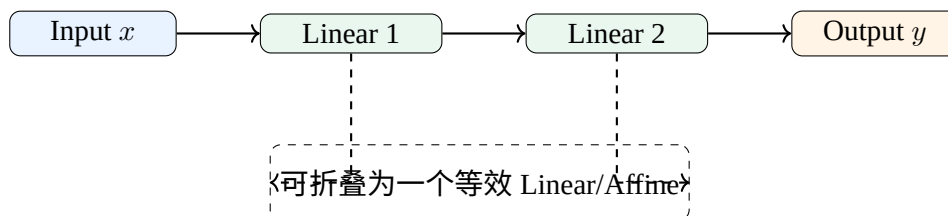
更准确地叫 affine map。

深度学习工程里，大家常把 Linear 层连 Bias 一起口语化叫“线性层”。

必须纠正的误区

Bias 不会制造神经网络需要的那种非线性。它允许平移，但多个 affine 层复合以后仍然是 affine。

8 【图解】为什么“层多”还不够



如果有 5 层、10 层，只要中间仍然没有非线性或其他不可折叠操作，仍然可以反复执行同样的合并。

9 为什么这限制了表达能力？

考虑一个极简单的非线性目标：

$$y = x^2$$

取三个点：

$$(-1, 1), \quad (0, 0), \quad (1, 1)$$

假设一条 affine 直线：

$$y = ax + b$$

为了通过 $(0, 0)$ ，必须：

$$b = 0$$

为了通过 $(1, 1)$ ，必须：

$$a = 1$$

于是这条直线只能是：

$$y = x$$

可在 $x = -1$ 时它给出：

$$y = -1$$

而目标是：

$$y = 1$$

因此一条 affine 函数不可能同时表达这三个点。

核心概念

如果多层 Linear 最终仍然只是一个 affine，那么它也无法突破这个限制。深度本身没有改变函数类别。

10 那非线性到底改变了什么？

先只做一个预告。下一讲会系统学习 Activation Function。

例如 ReLU：

$$\text{ReLU}(z) = \max(0, z)$$

如果网络变成：

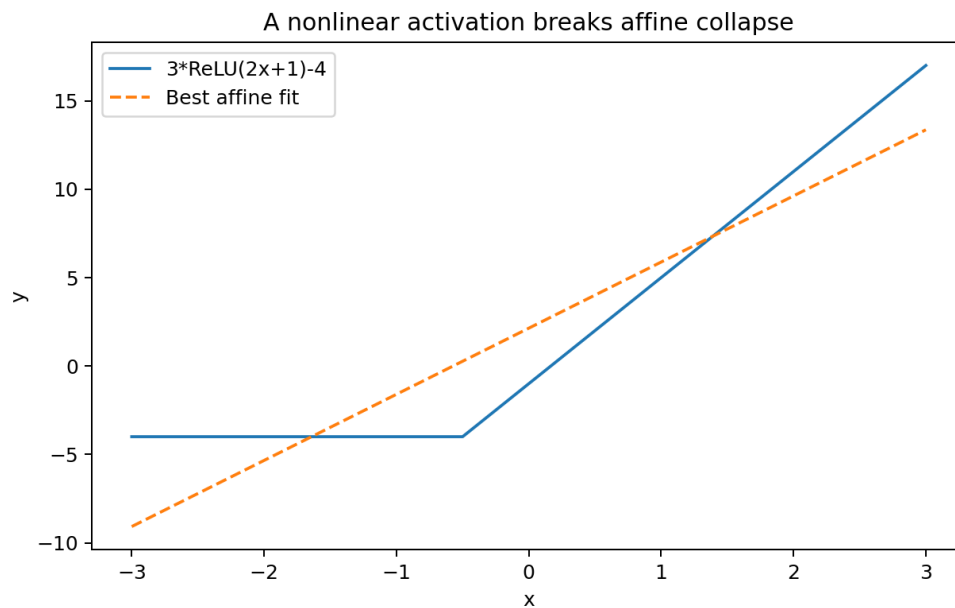
$$h = \text{ReLU}(2x + 1)$$

$$y = 3h - 4$$

那么：

$$y = 3 \text{ReLU}(2x + 1) - 4$$

它在 $2x + 1 = 0$ ，也就是 $x = -0.5$ 的地方出现“折点”。左边和右边斜率不同，因此不可能用一条全局直线精确替代。



“非线性”不等于必须画成光滑曲线

ReLU 每一段其实都是直线，但它整体不是一个单一 affine 函数。它是 piecewise linear（分段线性），因此依然属于深度学习所说的“非线性激活”。

11 【代码实验】用 NumPy 把结论跑出来

主实验文件：

```
lesson08_why_nonlinearity_experiment.py
```

最核心的折叠代码：

```
H = X @ W1 + b1
Y_two = H @ W2 + b2

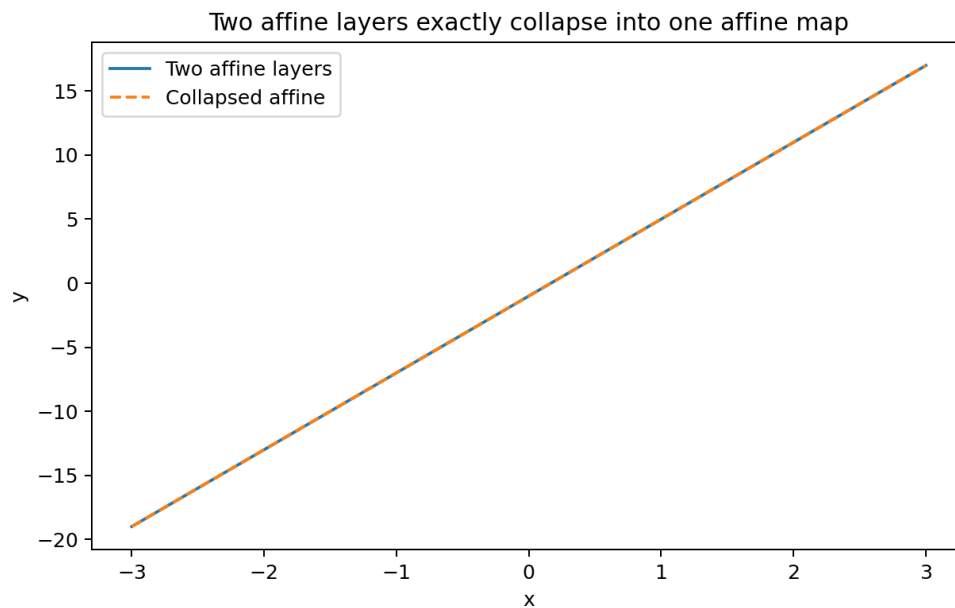
W_eq = W1 @ W2
b_eq = b1 @ W2 + b2
Y_one = X @ W_eq + b_eq

print(np.allclose(Y_two, Y_one))
```

实际输出：

```
完全一致: True
```

五层随机 affine 网络也验证：最大数值误差只有约 3.55×10^{-15} ，属于浮点误差范围。



两条线在图上完全重合，因为它们本来就是同一个函数。

12 【实验结果】加入 ReLU 后为什么折叠失败

实验让：

$$y = 3 \operatorname{ReLU}(2x + 1) - 4$$

然后用最小二乘找一条最接近它的直线：

$$y = ax + b$$

得到：

$$a \approx 3.7406, \quad b \approx 2.1395$$

但仍然有：

$$\text{最大绝对误差} \approx 5.0824$$

$$MSE \approx 6.2481$$

不是“优化还不够好”，而是函数类别本身不一样。

13 【常见误区】

13.1 误区 1：神经网络只要层数多就一定更强

不对。若都是可连续折叠的 affine 层，函数类别仍是 affine。

13.2 误区 2: Bias 已经让网络非线性

不对。Bias 只是平移。

13.3 误区 3: ReLU 是分段直线，所以它仍然算 linear

不对。它不能用一个全局 affine 映射表示，因此在神经网络语境中是非线性。

13.4 误区 4: 既然两层可折叠，那么训练两层和训练一层完全一样

也不对。函数表达类别可以相同，但参数化不同，训练路径、梯度动力学、隐式偏置都可能不同。

科研上非常重要的限定

“深层线性网络可折叠”说的是：固定参数后的输入-输出函数仍是 affine。

它不意味着：

- 优化过程完全一样；
- 参数数量一样；
- 中间表示毫无作用；
- 加入 bottleneck 后没有 rank 约束；
- 任意带 LayerNorm、Softmax、Dropout、门控、条件分支的网络都能折叠。

14 Bottleneck：多层 Linear 仍可能有结构约束

例如：

$$D_{in} = 100 \rightarrow D_{hidden} = 10 \rightarrow D_{out} = 100$$

虽然最终仍是一个线性映射，但等效矩阵的 rank 最多受中间 10 维 bottleneck 限制。

因此：

- 它没有获得非线性；
- 但参数分解方式可以对可表示的线性映射施加结构约束。

这类“表达类别”和“参数化结构”必须区分。

15 【与 LLM / Transformer 的关系】

15.1 Transformer FFN 为什么是 Linear -> Activation -> Linear

简化形式：

$$h = XW_1 + b_1$$

$$a = \text{GELU}(h)$$

$$y = aW_2 + b_2$$

如果把 GELU 删除：

$$y = (XW_1 + b_1)W_2 + b_2$$

那么 FFN 本身就可以折叠为一个 affine 层。

15.2 现代 LLM 常见 SiLU / SwiGLU

以后你会看到：

- GELU；
- SiLU；
- SwiGLU。

它们都在帮助模型产生不能被一个简单线性映射替代的变换。

15.3 不要误解为 “Transformer 的非线性只来自 FFN”

Transformer 中还有：

- Softmax Attention；
- LayerNorm / RMSNorm 等输入依赖操作；
- 门控结构；

这些也使整个网络不再是单纯线性系统。今天只聚焦 FFN 中 “为什么 Linear 之间需要 Activation”。

15.4 Agent Systems 为什么需要懂？

Agent Systems 研究经常比较不同 base model、adapter、projection、representation。看结构图时，你要能区分：

- 只是新增线性投影；
- 还是引入了真正新的非线性计算；
- 改的是模型表达能力、参数化约束，还是系统层行为。

16 【科研视角】Ablation 到底在测什么？

假设一篇论文提出新模块：

$$\text{Linear} \rightarrow \text{GELU} \rightarrow \text{Linear}$$

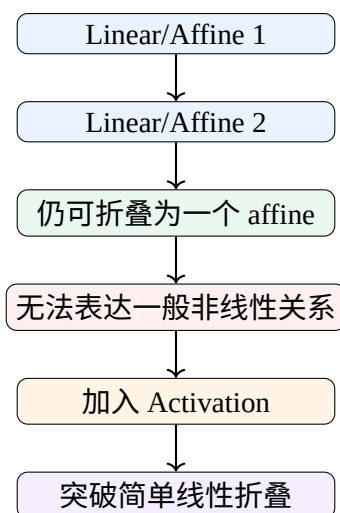
Ablation 把 GELU 删掉，性能明显下降。

你不能只写“GELU 有用”。更合理的问题是：

1. 删除非线性后，模块是否退化为单一 affine 映射？
2. 性能下降是否来自表达能力减少？
3. 参数量是否同时改变？
4. 是否存在训练稳定性差异？
5. 其他非线性函数能否替代？

这就是从“会看结果”走向“理解实验机制”。

17 本讲完整心智链



18 本讲检查点

如果下面问题都能回答，就可以进入 Activation Function

1. 为什么 $3(2x + 1) - 4$ 可以合并成 $6x - 1$ ？
2. 两层矩阵 affine 的 W_{eq} 、 b_{eq} 分别是什么？
3. 为什么 Bias 不等于非线性？
4. 为什么多层 Linear 无法表达 $y = x^2$ 的一般形状？
5. ReLU 为什么能阻止简单折叠？
6. “函数上可折叠”为什么不等于“训练过程完全相同”？
7. Transformer FFN 中间为什么需要 GELU/SiLU 等 Activation？

19 下一讲预告

第 09 讲将正式系统学习：

Activation Function：ReLU、Sigmoid、Tanh、GELU 与 SiLU

我们会逐个画曲线、手算输入输出，并解释为什么现代 Transformer 更偏好 GELU、SiLU、SwiGLU 一类激活。